

(PROGRESS REPORT)

Học phần: Operating Systems [Multimedia] - 20252

Mã số dự án (Project No): 25236

Tên dự án (Project title): Optimize pipe implementation to improve IPC throughput

Thiết kế (Design thinking): [Link Figma](#)

Quản lý tác vụ (Task management): [Link JIRA](#)

Thành viên (Members): Nguyễn Sỹ Tuấn, Lê Huy Sơn, Vũ Tiến Long

Giai đoạn báo cáo (Working period): Từ: 13/4/2026 Đến: 17/4/2026

1 Summary of latest progress

(Viết khoảng 200-300 từ để tóm tắt quá trình thực hiện trong giai đoạn này và tổng mức độ hoàn thành hiện tại)

Như đã nói ở tuần trước, chúng tôi đã đưa ra các tasks phải thực hiện trong tuần này là:

1. Tìm hiểu về pipes trong xv6 kernel
2. Phân tích file code pipe.c trong thư mục xv6-riscv/kernel

Để hoàn thành các tasks này, chúng tôi đã chia ra theo modules để hoàn thành từng task, cụ thể như sau:

+ **Đối với task tìm hiểu về pipes:**

1. Các trường trong pipe: SPINLOCK, nread, nwrite,...
2. Circular buffer.

+ **Đối với task phân tích file pipe.c:**

1. pipealloc()
2. pipeclose()
3. piperead()
4. pipewrite()

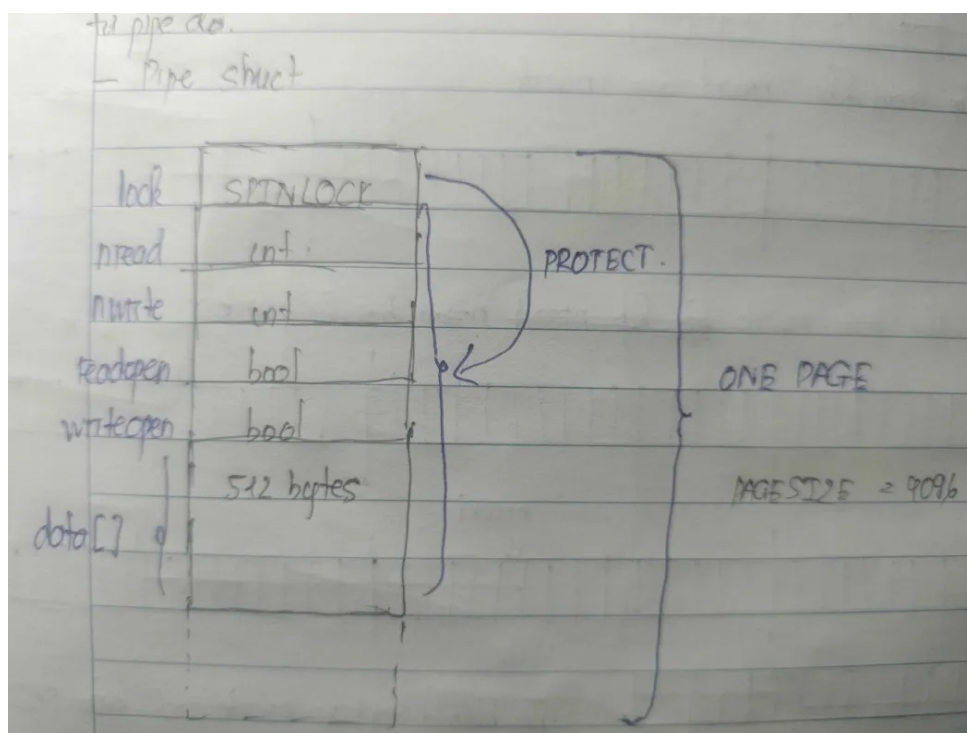
Việc phân tích từng task và modules sẽ được giải thích rõ trong mục **2 Review of technical activities** và trong file jira.

2 Review of technical activities

Activity 1: Phân tích về pipe

Trong UNIX, pipe giống như một file, ta có thể ghi dữ liệu vào nó và đọc dữ liệu từ nó. Tuy nhiên, file được lưu trữ trên đĩa, còn dữ liệu trong pipe hoàn toàn nằm trên bộ nhớ.

Pipe struct:



Hình 1: Pipe struct

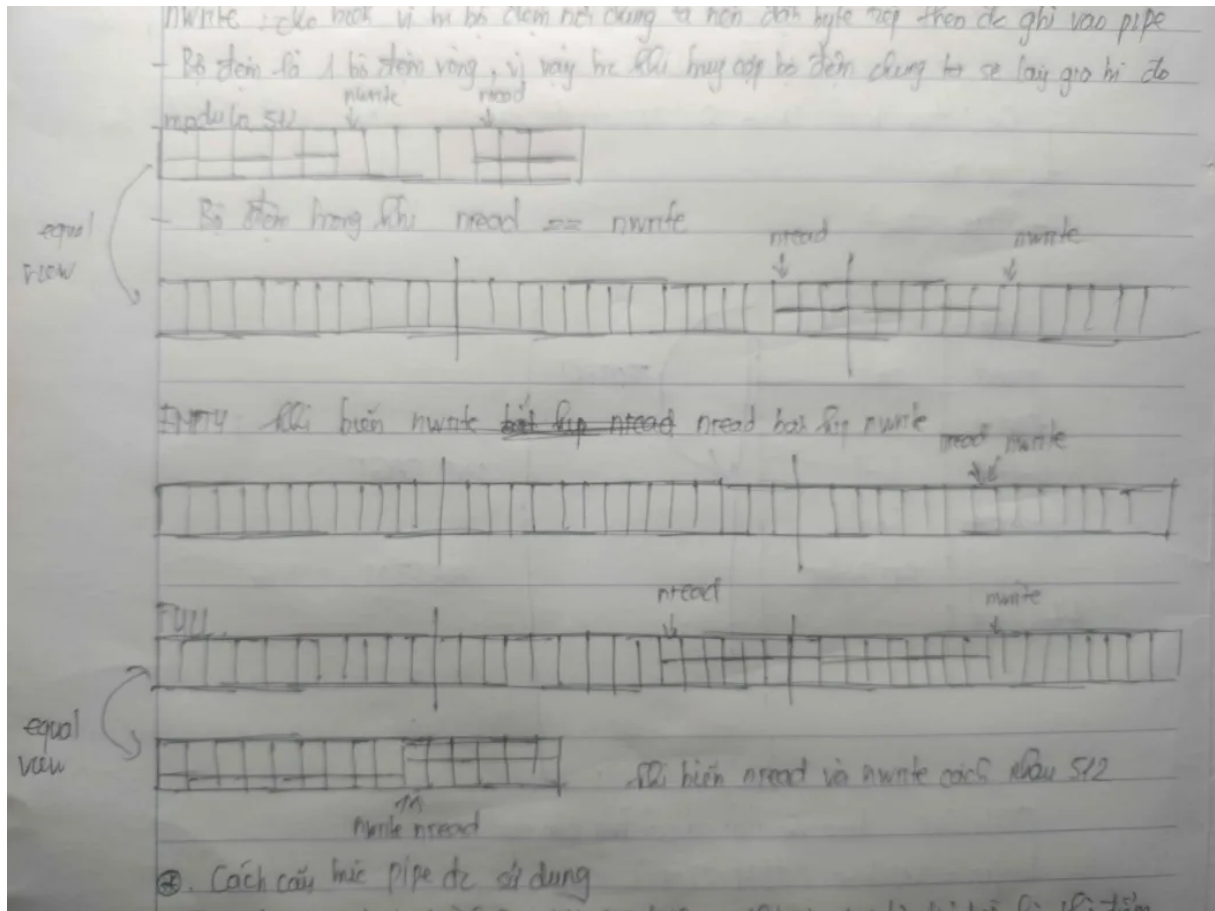
Các trường trong pipe:

- 1 SPINLOCK: bảo vệ tất cả các trường còn lại
- 2 nread, nwrite(int): chỉ số cho biết vị trí hiện tại của vị trí đọc và ghi trong bộ đệm
- 3 readopen, writeopen(bool): biến logic cho phép được đọc hoặc được ghi
- 4 data[]: bộ đệm 512 byte (PIPE_SIZE = 512)
- 5 empty field: trường trống, phân bổ inode, chứa hệ số thông tin của pipe

Bất cứ khi nào chúng ta cần tạo ra một pipe mới, HĐH sẽ gọi hàm `kalloc` và phân bổ 1 trang (PAGE_SIZE = 4096 Byte), khi hoàn thành, nó sẽ trả lại trang đó.

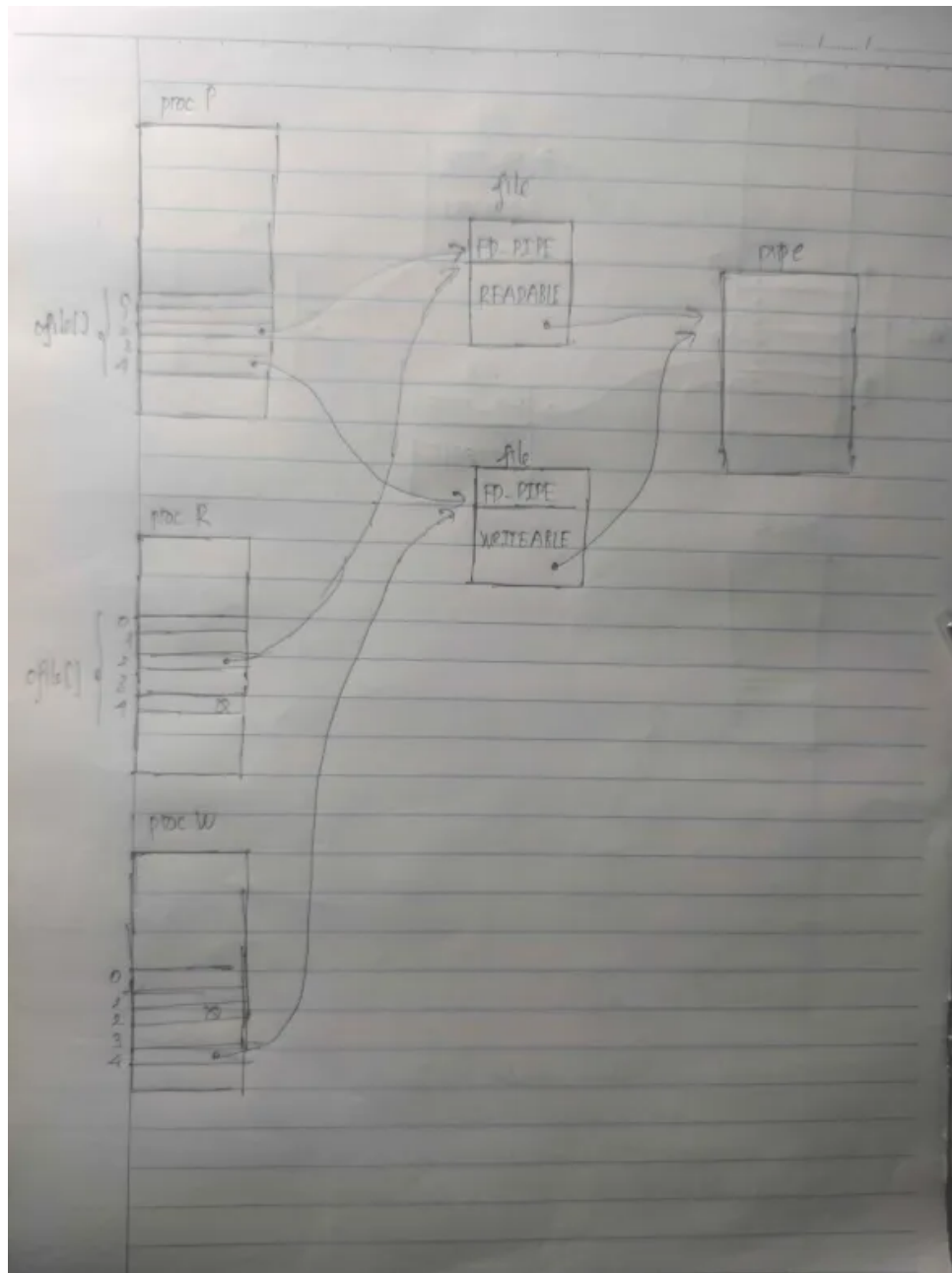
Các trường chiếm dữ liệu là khoảng 0.5K byte, trường trống còn lại là 3.5K byte đơn giản là không được sử dụng và lãng phí.

Pipe sử dụng bộ đệm vòng, vì vậy ta cần sử dụng phép toán modulo 512.



Hình 2: Pipe buffer

Cách cấu trúc pipe hoạt động: Cấu trúc pipe hoạt động như trên hình sau:



Giả sử có một tiến trình P. Một tiến trình có thể mở một số tệp tại bất kỳ thời điểm nào. User code tham chiếu đến các tệp riêng lẻ bằng cách sử dụng một bộ mô tả tệp (file descriptor). Bộ mô tả tệp là một số 0,1,2,... và nó được kernel sử dụng làm chỉ mục (index) của bộ mô tả tệp.

Trong cấu trúc của mỗi tiến trình, có một bộ mô tả tệp là một mảng `ofile[]` chứa các con trỏ đến các đối tượng tệp, mỗi file được bắt đầu với trường type (ở đây trường type mang type pipe(FD-PIPE)), và trong mỗi tệp chứa một con trỏ và con trỏ đó sẽ trỏ tới 1 đối tượng nào đó (pipe, inode,...).

Trong trường hợp của pipe, ta có 2 trường để đọc và ghi (READABLE và WRITEABLE), trường `readopen` và `writeopen` chỉ được mở khi có con trỏ trỏ tới pipe.

Giả sử tiến trình P gọi hệ thống pipe để tạo pipe và phân bố 2 đối tượng tệp và

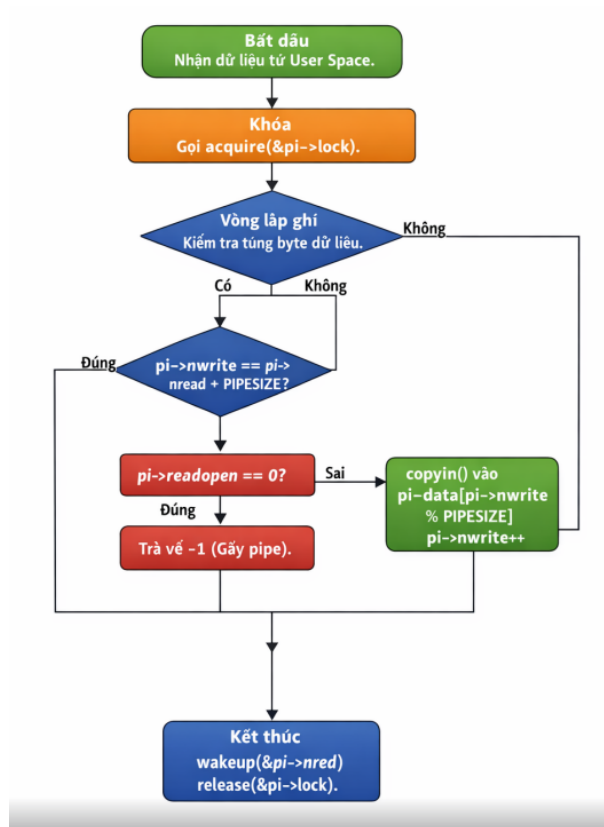
thiết lập các con trỏ. Sau khi `pipe()` system call được gọi, system call sẽ tìm trong tiến trình P các vị trí trong `ofile[]` và trả về các số ứng với 2 đối tượng tệp đó (2 và 4).

Tiến trình P `fork()` 2 children, là tiến trình R cho công việc đọc và tiến trình W cho công việc ghi. Do khi gọi `fork()`, tiến trình P sẽ tạo ra một tiến trình y hệt, do đó tiến trình R và W cũng giống như P, đều trỏ tới 2 file. Giả định là mỗi tiến trình làm đúng công việc của nó, nghĩa là tiến trình R sẽ set `ofile[]` tại chỉ mục mà nó trỏ tới `writelfile` là `NULL(ofile[4] = NULL)`, và tiến trình W sẽ set `ofile[]` tại chỉ mục mà nó trỏ tới `readfile` là `NULL(ofile[2] = NULL)`.

Tiến trình W có thể gửi dữ liệu đến tiến trình R bằng cách thực hiện một system call ghi trên tập tin cho bộ mô tả tệp chỉ mục 4, tương tự với tiến trình R có thể đọc dữ liệu từ tiến trình W.

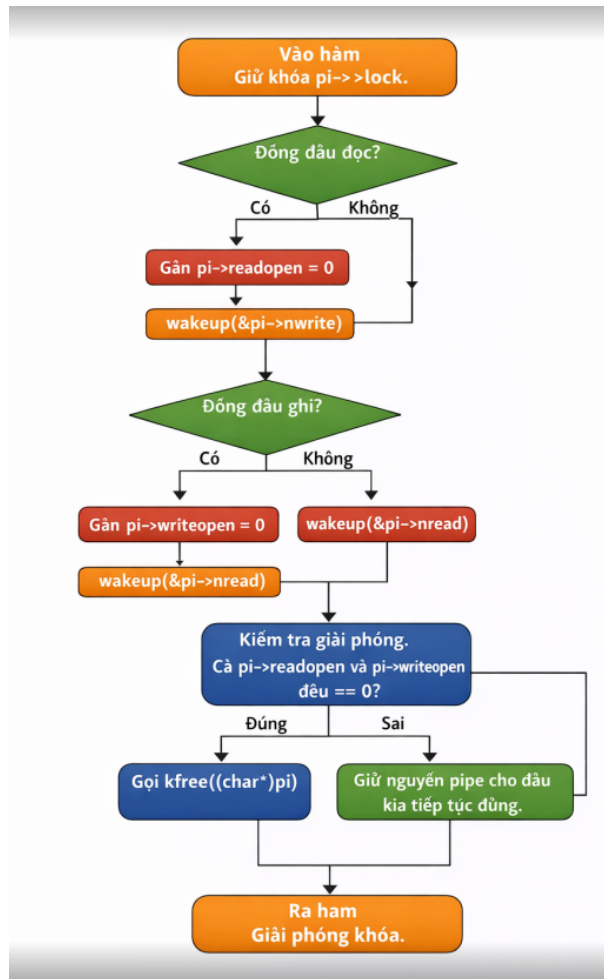
Activity 2: Phân tích các hàm trong `pipe.c`

- 1 `piperead()`: Hàm `piperead()` đọc dữ liệu từ buffer. Nó khóa tài nguyên bằng `acquire()`, kiểm tra nếu buffer rỗng thì `sleep()` chờ writer ghi dữ liệu. Khi có dữ liệu, nó đọc theo cơ chế FIFO, tăng biến `nread` và gọi `wakeup()` để đánh thức writer nếu trước đó buffer đầy.
- 2 `pipewrite()`: Chép dữ liệu từ không gian người dùng vào buffer của pipe. Hàm `pipewrite` sao chép `n` byte dữ liệu từ không gian người dùng (user space) vào bộ đệm của pipe trong kernel. Nếu bộ đệm bị đầy, tiến trình ghi sẽ phải "ngủ" để chờ tiến trình đọc lấy bớt dữ liệu ra.



3 pipeclose(): Giải phóng tài nguyên khi 1 đầu pipe bị đóng. Trong một hệ điều hành, 1 pipe có 2 đầu: 1 đầu để ghi(write), 1 đầu để đọc(read). Khi một tiến trình không còn dùng đầu đó nữa, nó gọi là pipeclose. Hàm này đảm bảo:

- Đánh dấu đầu đó đã đóng.
- Thông báo cho đầu kia biết (để không bị đợi vô hạn).
- Giải phóng vùng nhớ của pipe khi không còn ai sử dụng.



Activity 3:

3 Personal contribution

Member 1 (Nguyen Sy Tuan): Tìm hiểu về xv6 pipe.

Member 2 (Le Huy Son): Tìm hiểu về các hàm trong pipe.c.

Member 3 (Vu Tien Long): Tìm hiểu về các hàm trong pipe.c.

4 Developed code review

4.1 Algorithmic development

(Mô tả các thuật toán đã phát triển)

4.2 Source code enhanced and/or revised

(Các đoạn code đã được cải tiến hoặc chỉnh sửa)

4.3 Testing result

(Kết quả kiểm thử, có thể chèn ảnh minh họa bên dưới)

5 TODO

a) **List of remaining issues:**

b) **List of tasks to carry out in the next period:** Các task cần làm trong tuần sau: Viết được chương trình ví dụ về pipe trong xv6, tính toán được thông lượng trong pipe đó.

c) **Milestone:** *(Cần làm gì thêm để đạt được mốc tiến độ tiếp theo?)*